ChalkStream Group #62

Cameron Bodkin-Dick #101-272-935
Sandy Alzabadani #101-055-734
Eric Cui #101-237-617
Supervisor: Safaa Bedawi

Table of Contents

ChalkStream Group #62 1

1. Objectives 3
2. Background 4
3. What will we do? 5
4. How does ChalkStream relate to the degree program of each student? 17
5. A discussion of how the group, collectively, has the skills required to undertake the project. 18
6. Method(s) you are going to use in solving the problem and how they relate to knowledge you acquired in your degree program to-date. 19
7. A proposed timetable for completion of the project including major intermediate milestones. 19
8. Project risks and mitigation strategies. 23
9. Required special components and facilities that you require. 26
10. References 26

1. Objectives

The objective of ChalkStream is to design and implement a real-time classroom assistant that bridges the gap between traditional board-based instruction and modern digital learning environments. Our motivation is rooted in a recurring challenge, where students struggle to balance the tasks of transcribing lecture material while engaging meaningfully with the content. It is impractical for both at home and in person students to split their focus between these tasks. By engineering a system that automatically captures, processes, enriches, and organizes written board content in digital form, ChalkStream aims to enhance comprehension, accessibility, and long-term study outcomes.

This project takes an engineering approach by breaking down the ChalkStream lecture pipeline into measurable subsystems: vision capture, recognition, synchronization, live delivery, and AI-based enrichment. Each subsystem is tied to various functional requirements, as well as non-functional requirements that guarantee robustness, usability, and scalability in real-world classrooms. Collectively, these objectives define ChalkStream as not only a demonstration of technical feasibility but also a tool that meaningfully benefits the student learning experience.

Functional Requirements:

1. Real-Time Capture:
    a. The system shall acquire a continuous video feed from a webcam or connected camera.
    b. The system shall automatically detect and isolate the whiteboard or chalkboard region, applying perspective correction to produce a flat, readable image.
    c. The system shall provide instructor-level configuration for board type (whiteboard or chalkboard), enabling tailored enhancement protocols (brightness, contrast, sharpening, grayscale normalization).
2. Content Segmentation:
    a. The system shall segment captured board content into categories: text, mathematical expressions, and diagrams.
    b. Each segment shall be routed to specialized processing pipelines (OCR, equation recognition, or diagram reconstruction)
3. Text and Handwriting OCR (Optical Character Recognition):
    a. The system shall process handwritten text from the board into structured, searchable digital text.
    b. Recognized text shall be stored for lecture digitization, content indexing, and AI-assisted insights.
4. Mathematical Expression Recognition:
    a. The system shall recognize handwritten equations and convert them into LaTeX format.
    b. Converted expressions shall be linked to the lecture timeline for digitization, search, and enrichment tasks.
5. Diagram Recognition & Reconstruction:
    a. The system shall convert hand-drawn diagrams into clean, vectorized representations.
    b. Vectorized diagrams shall be stored and integrated into lecture playback for accurate reproduction.
6. Speech Transcription:
    a. The system shall capture instructor audio via a microphone.
    b. The system shall produce accurate transcripts, time-aligned with board content, for both live delivery and playback search.
7. Synchronization and Search:
    a. The system shall align text, equations, diagrams, and transcripts into a unified timeline.
    b. The system shall support keyword-based search with playback navigation to the relevant lecture segment.

8. AI-Based Enrichment:
    a. The system shall generate contextual summaries of lecture segments.
    b. The system shall provide language translation of captured text and transcripts.
9. Instructor Experience:
    a. The system shall enable instructors with board controls to facillilitate starting, modifying, and completing boards where heuristic detection would be inconceivable.
    b. The system shall allow instructors to control lecture visibility and student access (e.g., real-time, post-lecture, or restricted).
    c. The system shall enable quiz generation and facilitation from lecture content or instructor-supplied material.
    d. The system shall track student attendance and engagement, including time spent in the lecture session.
10. Student Interaction:
    a. The system shall allow students to create personalized notes by highlighting, annotating, and organizing lecture content.
    b. The system shall support exporting personalized lecture notes in common formats (e.g., PDF, LaTeX).

Non-functional requirements:

1. Performance: Maintain processing latency below 500 milliseconds per component, maintaining smooth performance during live use.
2. Scalability: Support simultaneous access by at least 50 concurrent students without service degradation.
3. Modularity: Design system components with replaceable recognition engines, allowing iterative improvement.
4. Reliability: Maintain 95% uptime during scheduled use, with self-recovery mechanisms in case of component failures.
5. Security and Privacy: Apply encryption in transit and at rest, with instructor-level controls for archiving, deletion, and sharing of captured data after the lecture ends.
6. Usability: Provide an accessible browser-based interface, so students and instructors can use the system intuitively without specialized training.

These objectives establish ChalkStream as both an applied research project and a practical engineering solution. Success will be measured through carefully chosen metrics, recognition accuracy, latency benchmarks, transcript error rates, and usability studies. In doing so, ChalkStream will not only demonstrate technical innovation but also show clear educational value, directly addressing the challenges faced in fast-paced, content-heavy lectures.

2. Background

Traditional lecture delivery relies heavily on handwritten notes from whiteboards and chalkboards. While it's a common strategy for real-time explanation, this teaching method still creates significant barriers for students who struggle to keep up with note-taking, particularly in technical or mathematically intensive courses. Additionally, it brings complexities for students who require asynchronous accommodations. The result is that students often miss key content or spend more time transcribing than understanding. Numerous digital solutions attempt to bridge this gap, but they fall short in specific ways.

Video conferencing platforms such as Zoom or Google Meet provide live video feeds and automatic transcription of speech. However, the board content remains a static video stream, unsearchable and not engaging for the student. Interactive classroom tools such as Wooclap or Mentimeter enable polls and engagement but do not integrate with lecture content. Hardware-driven products such as Mimio digitize board strokes but primarily focus on instructor presentation rather than empowering student comprehension. Similarly, Logitech Scribe improves board visibility for remote participants but leaves content as a passive video feed, with no structuring, searchability, or personalization. ShareTheBoard, another competitor, enhances legibility but does not restructure content into editable notes or align it with lecture transcripts.

The state of the art in this field emphasizes visibility and participation but rarely comprehension and organization. What sets ChalkStream apart is its deliberate focus on transforming board content into structured, usable study material. Instead of just seeing the board more clearly, students will have access to searchable notes, equation transcriptions, diagram reconstructions, synchronized transcripts, and AI-based enhancements. This elevates ChalkStream beyond digitization into the realm of adaptive learning.

3. What will we do?

The ChalkStream system is constructed as a pipeline of components. Each component is described here in terms of its purpose, inputs, outputs, and various implementation methods.

3.1.1 Image Capture

Description: Video feed from a camera in the lecture hall, pointed as uniformly and clearly as possible/convenient at the board.
Outputs: Video feed of physical board
Implementations:

- Integrated Webcam
    - The native camera on a laptop device (commonly 1080p resolution).
- USB Connected Camera
    - A USB camera, traditionally on a tripod that can be separate from the device and positioned towards the board (commonly higher resolution 1440p+)
- Overhead lecture camera
    - A permanent camera affixed to the top of some lecture halls with Bluetooth or USB connection for instructors

3.1 Frame Preprocessing

Description:

Detect the board area, correct perspective, and keep writing visible even when the instructor stands in front of it.

Inputs:

Video feed of physical board.

Outputs:

Rectified images of the board surface.

Primary Implementations:

1. **OpenCV Homography and Edge Detection for Perspective Correction. [1]**
    - Open Source Computer Vision Library toolkit for C++, Python, C
    - OpenCV provides image processing primitives (edges, corners) and homography transforms to unwrap a slanted/offset board back into a rectangle.
    - Open for research and commercial use

Python + OpenCV

- **Grayscale Conversion:** cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
- **Noise Removal / Smoothing:**
    - Gaussian blur: cv2.GaussianBlur(image, (kernel_size, kernel_size), 0)
    - Median blur: cv2.medianBlur(image, kernel_size)
- **Thresholding / Binarization:**
    - Simple threshold: _, binary = cv2.threshold(gray, thresh_value, 255, cv2.THRESH_BINARY)
    - Adaptive threshold: cv2.adaptiveThreshold(...)
- **Edge Detection:**
    - Canny edge detection: edges = cv2.Canny(gray, lower_thresh, upper_thresh)

In ChalkStream, OpenCV will serve as the foundation of the image pre-processing pipeline. It will be used to detect and crop the board region through contour analysis, followed by perspective correction with image transforms so that angled views are resolved into a flat, rectangular images. Glare and lighting inconsistencies will be reduced using brightness equalization and inpainting, while gently enhancing stroke contrast for downstream OCR and diagram recognition. Together, these steps ensure that any processed video frame delivers a clean, normalized representation of the board, making recognition more accurate and reliable.

Adaptive Background Modeling is the proposed approach to reliably capture new images from the board while the presenter is obstructing the board. It gradually learns the board as the background while filtering out the moving professor. The learning rate is kept low enough so that newly written text is absorbed into the background and remains visible, but not so fast that the professor becomes part of the background. After background subtraction, apply morphological operations to remove noise, and optionally use thresholding to enhance the contrast of the text against the board. This pipeline ensures the professor is effectively removed while the board text remains sharp for recognition.

Initialize background model

- Use OpenCV's cv2.createBackgroundSubtractorMOG2() or cv2.createBackgroundSubtractorKNN().

Tune learning rate

- Set a low learning rate so that newly written text becomes part of the background.
- Ensure it's not too low, otherwise lighting changes will break the model.

Apply background subtraction

- For each frame, call fgmask = subtractor.apply(frame, learningRate=…).
- This produces a mask where the moving professor is foreground and the board + text remain background.

Clean up with morphological operations

- Use cv2.morphologyEx(fgmask, cv2.MORPH_OPEN, kernel) to remove noise.
- Optionally use cv2.MORPH_CLOSE to fill small gaps in the text.

Enhance text visibility

- Apply thresholding (cv2.threshold) to improve contrast between the board and text.
- Optionally apply smoothing to reduce leftover shadows.

Reconstruct the clean board

- Use the updated background model as the "board-only" frame.
- The professor is filtered out, while the text stays visible.

Feed into OCR

- Pass the processed frame to an OCR engine (e.g., Tesseract) for text extraction.

Alternatives Approaches (if primary implementation proves to have shortfalls)

Method	Description	Advantage	Limitation
ArUco fiducial markers for robust board localization.	ArUco are simple black and white square tags that the camera can recognize to infer the board's corners and pose. Physical printed indicators to guarantee the correct board perspective.	Persistent board image correction and isolation.	Physical printed cards that the intructor needs to install themselves.
Temporal Median Filtering (Long-Term Frame Averaging)	Build a background by taking the median of several consecutive frames at each pixel location over time. Over time, volatile objects (like the instructor) are statistically filtered out, leaving the static board.	No learning rate parameter, simple and effective for slowly changing scenes, and easy to combine with OpenCV (few lines of code)	Requires a buffer of many frames. Slow response to new writing, recent chalk strokes may take a few seconds to appear.

3.2 Content Segmentation

Description:

This part of the system finds and separates all the writing on the board so we can tell which areas are text, equations, or diagrams. To do this, we use a mix of OpenCV tools to detect where writing appears and PaddleOCR to check whether the writing actually looks like handwritten text. We then apply a set of rules that consider how confident PaddleOCR is and what kinds of symbols appear in each area to decide which type of content it is.

Inputs:

Perspective-corrected board images (output of Section 4.1 Board Capture).

Outputs:

Boxes drawn around each detected area of writing, labeled as Text, Equation, or Diagram, with a confidence value for each one. These labeled regions are then passed to the next parts of the pipeline for detailed handwriting OCR, math recognition, or diagram vectorization

Primary Implementation

1. **OpenCV Toolkit configured for Candidate Region Detection**:

We start by using OpenCV functions like grayscale conversion, thresholding, contour detection, and a few morphological filters to highlight areas on the board where chalk or marker strokes appear. This helps remove background noise and glare so we only keep the parts that actually contain writing. The result is a set of candidate regions that likely contain words, equations, or drawings. [1]

2. **Text Confidence Detection (PaddleOCR DBNet model)**

Each candidate region is then passed into PaddleOCR's DBNet detector, which gives a confidence score showing how confident that region contains real handwriting. We use a lightweight pretrained model, such as DBNet-Mobile, so it can run quickly on normal hardware. PaddleOCR can also include a small recognition head that reads short sequences of characters. We use this optional step when we want more detail about the symbols or tokens inside each detected area.

Optional: If needed, the model can be fine-tuned using real annotated handwritten lecture board samples to better adapt to classroom lighting and stroke patterns if it's not consistent. [4]

3. **Classifying the Type of Writing**

Once we have the confidence score and any recognized text, we use simple logic to decide what type of content it is. Regions with high confidence and mostly letters or words are labeled as Text. Regions with a noticeable amount of mathematical symbols, such as =, √, ∫, Σ, or π, are labeled as Equations. Regions with low confidence or no recognizable characters are labeled as Diagrams and sent to the diagram vectorization stage later in the pipeline. This rule-based method gives us an efficient way to tell different kinds of handwritten content apart without having to train a new model.

Design VS Configuration for Implementation

Step	Description	Category
OpenCV Toolkit configured for Candidate Region Detection	Uses existing OpenCV functions and tuned thresholds to find where writing appears	Toolkit configuration
Text Confidence Detection	Uses pretrained handwriting models to get rudimentary scores with the option to train this lightweight model on blackboard lecture video if needed [4]	Configured / Fine tuned if needed [4]

Classifying the Type of Writing	Custom rules that combine confidence and symbol ratio to decide text, equation, or diagram	Designed

Alternative Approaches

Method	Description	Advantage	Limitation
CRAFT [2]	A CNN trained on natural-scene text (character heatmaps) is useful for text detection	Accurate for Latin text; open source	Not trained on math symbols or chalkboard style; limited handwriting robustness
YOLOv8 Custom 3-Class Detector	Train end-to-end on text, equations, and diagram labels	Very fast (15 ms per image)	Requires ~500 labeled frames, GPU training needed
Chosen OpenCV + PaddleOCR	Combines OpenCV for region detection with PaddleOCR for handwriting confidence and rule-based classification.	Lightweight, explainable, runs in real time, and uses pretrained models that already work for handwriting	Requires small adjustments for board contrast and relies on simple rules for equation separation.

3.3 Handwriting OCR

Description:

This part of the system converts handwritten words on the board into editable and searchable digital text. After the content segmentation step, label regions as text, these cropped text regions are passed into a handwriting recognition model based on TrOCR (Transformer-based Optical Character Recognition). TrOCR is a modern image to text model that uses an image encoder and a transformer language decoder to output text directly from handwritten input. We plan to use an existing pretrained TrOCR model and fine-tune it on samples that reflect classroom whiteboard/blackboard handwriting to improve its accuracy.

Inputs:

Cropped handwriting regions labeled as Text by the segmentation step

Outputs:

Recognized text strings with confidence scores and bounding box references for placement on the digital canvas.

Primary Implementation:

1. **Pretrained Model Setup**

We start from a pretrained TrOCR model available through the Hugging Face Transformers library. These models have already been trained on large handwriting datasets such as IAM and RIMES, so they should perform well on general handwriting. Fine-tuning will focus on adjusting the model to handle marker and chalk writing styles, which often have lower contrast and thicker strokes than paper handwriting.

2. **Fine-Tuning on Whiteboard Samples**

To adapt TrOCR to our domain, we plan a lightweight fine-tuning step using real board images.

Board frames from datasets like LectureMath [3] (prefiltered for text crops only) and LectureVideoDB [4] will be used. Around 500-700 examples should be enough for domain adaptation without requiring heavy retraining. This fine tuning step helps the model adjust to variations in lighting, handwriting thickness, and board color. Notably, the LectureVideoDB [4] dataset has been shown to outperform the IAM dataset for whiteboard and blackboard handwriting, so including it in fine-tuning is expected to significantly improve accuracy on classroom data.

3. Post-Processing and Integration

Once TrOCR produces text output, we clean and format it by correcting spacing and joining split characters. The recognized text is stored with its bounding box coordinates so it can be reconstructed on the digital board later or aligned with the transcript in the synchronization stage.d

Design VS Configuration for Implementation

Step	Description	Category
Pretrained TrOCR model	Uses an existing pretrained transformer-based OCR model from the Hugging Face library	Configured
Fine-tuning with Whiteboards/blackboard samples	Uses pretrained handwriting model that can be fine-tuned to improve blackboard or whiteboard detection	Fine-tuned [4]
Classifying the Type of Writing	Custom rules that combine confidence and symbol ratio to decide text, equation, or diagram	Designed

Alternative Approaches

Method	Description	Advantage	Limitation
PaddleOCR (Handwriting Model)	CNN + RNN-based OCR system developed by Baidu, with some pretrained handwriting models and flexible configurations.	Easy to deploy, supports multi-language text, integrates naturally with the DBNet detector already used in segmentation.	Slightly less accurate on free style handwriting, and weaker at recognizing cursive or connected text
SATRN (Spatial Attention Transformer Network)	Transformer based handwriting recognizer that uses spatial attention to focus on local regions of handwriting.	Performs well on irregular, slanted, or stylized handwriting, and strong accuracy on IAM and CVL datasets.	Complex model requiring GPU acceleration and larger labeled data for effective training, and harder to fine-tune quickly.
TrOCR (Chosen Approach)	Transformer model pretrained by Microsoft on large handwriting and printed text datasets. It directly converts images into text tokens.	High accuracy, handles cursive and uneven handwriting, easy to fine-tune using lecture datasets like LectureMath [3] and LectureVideoDB [4].	Requires GPU for fine-tuning and can be slower than lightweight CNN models during inference.

3.4 Math OCR

Description:

This part of the system identifies and interprets handwritten mathematical equations on the board, converting them into structured markup formats (LaTeX). Unlike text OCR, math OCR must recognize not only the characters but also their spatial arrangement, for example, fractions, superscripts, subscripts, and matrices. This makes it a much more complex problem, as the model must infer mathematical structure as well as symbol identity.

Inputs:

Cropped equation images labeled as Equations by the content segmentation step.

Outputs:

Structured LaTeX or MathML expressions with confidence scores and bounding box references for their position on the digital board.

Primary Implementation:

1. **Base Model (Im2LaTeX)**

The main model used with be Im2LaTeX, a neural network that converts images of equations into LaTeX code using an encoder-decoder architecture. It uses a CNN encoder to extract spatial features from the equation image and an attention-based transformer decoder to predict a sequence of LaTeX tokens. This model has been trained on large datasets such as Im2Latex-100K and provides a strong starting point for handwritten equation recognition.

2. **Two-Stage Fine-Tuning with CROHME** [6] **and LectureMath** [3]

To adapt the base model to handwritten classroom equations, we use a progressive two-stage fine-tuning approach:

Stage 1 Structural Learning (CROHME [6]) .inkml (=> .png) => .tex

The model is first fine-tuned using the CROHME 2023 dataset, which contains thousands of handwritten mathematical expressions paired with ground-truth LaTeX labels. This stage helps the sequence model learn symbol and layout structure, improving accuracy of the output equations. The CROHME data, originally stored in InkML format, is converted into grayscale images to align with Im2LaTeX input requirements.

Stage 2 - Environmental Adaptation (LectureMath [3]) .png => .tex

After learning structural grammar from CROHME, the model is further fine-tuned using LectureMath, which contains real whiteboard equations written by instructors and annotated with LaTeX transcriptions. This step teaches the model to handle real-world challenges such as glare, uneven lighting, marker thickness, and imperfect handwriting, improving robustness in classroom environments.

3. **Post-Processing and Integration**

Generated LaTeX is validated, simplified, and passed to a renderer such as KaTeX or MathJax for visualization on the digital board. The final outputs can also be stored as .tex or .svg files for later export.

Design VS Configuration for Implementation

Step	Description	Category
Im2LaTeX base model	Pretrained equation-to-LaTeX transformer	Configured
Stage 1 fine-tuning with LectureMath	Domain adaptation for whiteboard and chalkboard conditions	Fine-tuned [6]

Stage 2 fine-tuning with CROHME	Symbolic LaTeX structure and handwritten expression recognition	Fine-tuned [3]
Post-processing cleanup	Custom logic for syntax and display	Designed

Alternative Approaches

Method	Description	Advantage	Limitation
CROHME Baseline System (MyScript/ WIRIS)	Traditional symbol-based recognizers	Tuned for handwriting	Harder to integrate, less control, weaker at handwriting
MathPix API	Closed-source commercial math OCR engine	Highest accuracy, handles complex layouts	Paid API, not trainable
Vision Transformer (ViT) encoder	Replaces CNN with transformer embeddings	Better global context	Computationally heavy

3.5 Diagram Vectorization

Description:

This part of the system takes the regions labeled as Diagrams from the content segmentation step and converts the hand-drawn lines, shapes, and arrows into clean, scalable vector graphics.
The goal is to recreate the instructor's drawings digitally while preserving their layout and proportions. Unlike text or equations, these regions rarely contain identifiable characters, so a full OCR model is unnecessary. Instead, this stage relies on a series of algorithmic image-processing methods that detect edges, geometric primitives, and spatial relationships to rebuild the drawing as a structured set of lines, curves, and shapes.

Inputs:

Cropped diagram regions identified during the content-segmentation stage.

Outputs:

Vectorized representations of the original diagrams stored as SVG or JSON objects, containing primitives such as lines, curves, and shapes.

Primary Implementation:

1. **Edge Extraction (OpenCV)**

Using OpenCV functions such as Canny edge detection, we extract the boundaries of all visible strokes within each diagram region. This step converts the raw image into a binary edge map that isolates only the drawn lines, removing most background texture or board artifacts.

2. **Contour Approximation and Line Grouping**

Contours are traced from the edge map and simplified using polygonal approximation to reduce noise and small jitter caused by handwriting. Nearby line segments are grouped based on angle and distance thresholds, allowing continuous strokes to remain connected. The output at this stage is a clean set of coordinates representing each stroke path.

3. **Vectorization (Potrace)**

The cleaned binary mask is passed through Potrace, a lightweight open-source algorithm that converts bitmap regions into smooth Bézier curves. The output is stored as SVG paths or JSON coordinate data, which can then be scaled or recolored on the digital board. No machine learning model is required, this is a deterministic, geometry-based process that ensures low latency and consistent reproduction.

Design VS Configuration for Implementation

Step	**Description**	**Category**
Edge detection	Uses OpenCV functions (Canny or Sobel) to extract strokes	Configured
Contour tracing and simplification	Custom algorithm for grouping and smoothing connected strokes	Designed
Vectorization (Potrace)	Converts simplified bitmap into Bézier curves	Configured
Coordinate preservation	Maintains stroke positions relative to board coordinates for digital overlay	Designed

Alternative Approaches

Method	**Description**	**Advantage**	**Limitation**
Geometric detection algorithm	Finds straight lines and major contours in noisy handwritten images	Fast, deterministic, and works well on chalk or marker drawings. Good for boards with structured content (boxes, arrows, grids)	Only detects straight lines, not curves or organic shapes. Doesn't capture stroke width or irregular handwriting.
Chosen: Algorithmic Vector Tracing (OpenCV + Potrace)	Lightweight, deterministic reproduction of board drawings	Fast, explainable, no training needed	Captures geometry only, not meaning

3.6 Speech-to-Text (ASR)

Description:

This component transcribes the instructor's speech into accurate, time-aligned text that can be synchronized with the captured board content. The transcription serves as both a standalone transcript and a semantic anchor for lecture search, summarization, and translation. The main challenge is to achieve high accuracy in real classroom conditions where background noise, reverb, overlapping speech, and diverse accents are common.

Inputs:

Audio captured from the classroom microphone (wired, wireless, or optionally laptop-integrated)

Outputs:

Text transcript of instructor speech, divided into time-stamped segments.

Primary Implementation:

1. **Base Model: OpenAI Whisper Transformer ASR [7]**

We will employ Whisper Small or Whisper Medium (depending on available hardware) as the primary model for automatic speech recognition (ASR). Whisper is a transformer-based encoder decoder model trained on hundreds of thousands of hours of supervised data. It performs well even in noisy and accented speech conditions ideal for lecture halls.

- Inference pipeline: Audio is chunked into 30-second segments, processed in FP16 precision, and decoded with beam search to maximize confidence.
- Timestamp alignment: The model outputs timestamps for each token, enabling synchronization with board events.

2. **Post-Processing and Integration**

- Text normalization converts numbers, symbols, and shorthand into readable math-friendly text.
- Transcripts are aligned with lecture timestamps and stored in JSON format.
- The final transcript is fed into the Multimodal Fusion stage to align with detected board content for full lecture searchability.

Alternative Approaches

Method	Description	Advantage	Limitation
Google Cloud Speech-to-Text API	Cloud-based streaming ASR with speaker diarization and automatic punctuation	Minimal setup, scales easily	Requires network connection; recurring cost; limited customization
Kaldi Toolkit	Traditional modular ASR pipeline (feature extraction, acoustic + language models)	Fully open-source, customizable	Complex to configure and less robust to noise than transformer-based models

The ASR module continuously listens during lecture capture, streams transcript updates to the synchronization service, and tags each phrase with precise timestamps.
This allows ChalkStream to:

- Link spoken explanations to corresponding equations or diagrams.
- Enable searchable playback (e.g., students can jump to “Laplace transform” moments).
- Support downstream AI modules such as automatic summarization, translation, and quiz generation

3.7 Multimodal Fusion and Structuring

Description:

This component combines all outputs from the recognition stages into one organized lecture data structure. Its purpose is to synchronize and store the text, equations, diagrams, and transcripts that were generated by earlier components. The structured data is used both for live viewing during the lecture and for later playback or export. By maintaining a single timeline that contains all recognized information, ChalkStream ensures that the lecture can be replayed, searched, and reviewed in an interactive and meaningful way.

Inputs:

Recognized text, equations, and diagrams with position/size data, transcripts, and various metadata about the current board session.

Outputs:

A unified, time-stamped lecture structure containing all recognized entities (text, math, diagrams, speech)

Primary Implementation:

1. **Data Collection and Synchronization**

Each recognition component (OCR, Math OCR, Diagram Vectorization, and ASR) outputs structured data containing both spatial and temporal information. The fusion process combines these outputs into a single timeline by matching timestamps and aligning them with the corresponding board session.

2. **Data Structuring**

The combined data is stored as an organized object that includes each board segment, its written content, and any related transcript portions. Each segment has:

- A timestamp range indicating when it was active
- Lists of recognized text, equations, and diagrams
- Transcript entries that occurred during the same period

This structure makes it possible to view or replay a lecture while preserving the sequence of writing and speech.

3. **Storage and Delivery**

The finalized lecture data is serialized into JSON and stored on the server for later use. During a live session, the data is streamed to connected clients, where it is rendered in the web interface. Each client reconstructs the board timeline locally, allowing smooth playback and interaction even with limited bandwidth. After the lecture ends, the same structured data is archived for review, note-taking, or export.

3.8 AI Services (Summarization, Translation, Q&A)

Description:

This component provides additional functionality that enhances how students interact with recorded or live lectures. Once the structured lecture data has been created, the AI Services module uses it to generate summaries, translations, and question-answer responses. These features are built on top of the existing recognition outputs and data structure rather than being part of the core processing pipeline. The goal is to improve accessibility and comprehension. Students can review condensed versions of lectures, translate material into their preferred language, or ask questions about specific topics and receive context-based answers.

Inputs:

Structured lecture data produced from the fusion module

Outputs:

Summaries of lecture sections, translated text / transcripts, answers linked to lecture content.

Primary Implementation:

1. **Summarization**

The summarization process uses pre-trained large language models to generate concise descriptions of lecture sections or topics. The structured data from each board segment, along with its transcript, is fed into the summarization model to produce short, meaningful summaries. These summaries help students quickly review key ideas locate specific material without rewatching the entire lecture.

2. **Translation**

For translation, the lecture text and transcripts are processed through an open-source neural translation model such as MarianMT. This allows students to switch between languages while maintaining technical accuracy.

3. **Question and Answer Generation**

The Q&A service allows students to ask questions about lecture content. When a query is received, the system searches the structured lecture data for relevant segments and passes them to a large language model. The model then generates a focused answer and includes references to the specific lecture segment or board frame it was derived from.

The AI Services module runs after all recognition and structuring steps have completed. It connects directly to the lecture database and operates on finalized data. During playback, students can toggle summaries, translate displayed text, or ask questions through the web interface. The AI results are cached and linked to timestamps so that summaries, translations, and answers remain consistent between users and across sessions.

3.9 Storage and Delivery

Description:

This component manages how all processed lecture data is stored, accessed, and delivered to users. Once the recognition and fusion stages are complete, the structured lecture data, media files, and metadata must be saved in a reliable and scalable format. The storage layer ensures that the system can support both real-time streaming during lectures and on-demand playback afterward.

Inputs:

Structured lecture data from the fusion module

Outputs:

Securely stored lecture data, real-time data streams for live sessions, API and web endpoints for lecture retrieval and playback

Primary Implementation:

1. **Data Storage Architecture**

All data produced by the system is stored using a combination of relational and object storage.

- PostgreSQL Database: Handles metadata such as timestamps, board content references, transcripts, and user permissions.
- Object Storage (e.g., Azure Blob or Amazon S3): Stores large media files including video frames, audio, and diagram exports.

2. **Real-Time Delivery**

During a live lecture, the system streams structured data and processed board updates to clients through WebSocket connections. This keeps students' devices synchronized with the instructor's view while using minimal bandwidth.

3. **Post Lecture Access**

After the lecture ends, all data is stored in the same structured format and made accessible through the ChalkStream web portal. Students can search by keyword, filter by board, or replay specific segments. The system retrieves data through REST API endpoints.

3.10 Host server

Description:

The host server provides the main environment where all system components connect, communicate, and operate. It manages the flow of data between the processing pipeline, the database, and the web interface used by instructors and students. The server handles authentication, session management, file access, and user permissions while maintaining stable performance during both live and recorded lecture sessions.

Inputs:

Requests from the web application for live or stored lecture data, structured lecture data, media files, and AI results from the backend pipeline, instructor and student login credentials and session requests

Outputs:

Dynamic web pages and API responses delivered to users, real-time data streams for active lecture sessions, logged activity and performance data for monitoring

Primary Implementation:

1. **Server Framework**

The web server will be built using a python based backend such as Flask, chosen for performance and simple integration with AI and data processing libraries. The server exposes REST API endpoints for data retrieval and WebSocket endpoints for live streaming during lectures.

2. **User Management and Security**

User accounts are divided into two roles, instructor and student.

- Instructors can create, start, and manage lectures, as well as review stored sessions
- Students can join live session or access stored materials for playback and note-taking.

3. **Integration with Storage and Database**

The server connects directly to the PostgreSQL database and object storage. It retrieves structured lecture data, media files, and AI-generated outputs when requested by users. This integration allows instructors to modify lecture access settings and students to interact with the same dataset used during the live session.

4. **Client Communication**

The server maintains persistent WebSocket connections with client devices. This ensures that updates such as new board content, transcripts, or annotations are transmitted instantly. For stored sessions, the server sends cached lecture data in smaller packets to reduce load times and network usage.

The host server is the heart of ChalkStream. It links all backend processing components to the user interface and manages how data moves between them. During live sessions, it handles streaming, synchronization, and user control. After the lecture, it serves stored materials from the same system, ensuring a consistent and reliable experience for both instructors and students.

Together, these components form the complete ChalkStream lecture processing pipeline, transforming live classroom content into an organized, interactive, and accessible digital learning experience.

4. How does ChalkStream relate to the degree program of each student?

ChalkStream involves software engineering techniques and skills that have been developed, acquired and practiced throughout our degree. The project integrates multiple areas of the curriculum including software design, data structures, databases, artificial intelligence, user interface design, and systems integration.

The design and implementation of ChalkStream follow the structured process of the Software Developmental Life Cycle. This includes requirement analysis, system design, implementation, testing, and deployment. Each phase builds on software engineering fundamentals such as modularity, maintainability, and version control, which are essential to project development.

The system architecture reflects concepts taught in courses such as Software Architecture, Data Communication, and Real-Time Systems, where students learn how independent modules interact through defined interfaces and data protocols. Concepts from Database Systems are applied in designing the lecture data storage layer, while principles from Computer Networks and Distributed Systems are used to implement client-server communication for live lecture delivery.

ChalkStream represents a complete end-to-end engineering problem that involves system integration, real-time processing, and client-server architecture. It allows the group to combine their specialized knowledge into a single cohesive system that provides a benefit to both instructors and students. Through this project, each student will demonstrate not only technical competency but also teamwork, communication, and project management skills expected of graduating software engineers.

5. A discussion of how the group, collectively, has the skills required to undertake the project.

All members of the ChalkStream group are working towards their last year of their software engineering program. Degree wise, the group members share similar skills regarding courses taken and year standings. The group has strong programming skills in Python, Java, and SQL, which are essential for developing and integrating ChalkStream's components. Python is used for backend processing and AI model interaction, while Java experience supports system logic and data structure organization. Coursework in data management and software architecture provides the necessary background for implementing databases, APIs, and client-server systems that make up ChalkStream's infrastructure.

Some members have completed or are currently enrolled in courses such as Artificial Intelligence, Machine Learning, and Software Validation, which provide the theoretical and practical background to understand the models used in handwriting, math, and speech recognition. Others bring additional internship experience in database design, user interface development, and data visualization, which are directly applied in building the web platform and structured storage layers.

For expertise in computer vision techniques, like frame differencing, background modeling, and contour analysis is critical for the preprocessing and board detection steps. Our group will be utilizing learning platforms such Kaggle and Coursera to deepen our understanding in ML, computer vision and image processing. As well as, referring to documentation, academic journals, research articles and library references. This approach allows us to stay updated with the latest methodologies and best practices, ensuring that our project is informed by both practical applications and current research findings.

Together, the ChalkStream team has the technical ability, practical experience, and communication discipline required to successfully develop an integrated, real-time classroom assistance system that demonstrates the full range of competencies expected from graduating software engineers.

6. Method(s) you are going to use in solving the problem and how they relate to knowledge you acquired in your degree program to-date.

The methods used to build ChalkStream are based on the core principles taught throughout the Software Engineering program. The project follows the SDLC, including requirement analysis, design, implementation, testing, and deployment. Each phase applies techniques learned in software design, project management, and validation courses.

The backend and data systems use relational database concepts such as normalization, indexing, and query optimization from Database Systems. Client-server communication applies knowledge from Data Communication and Distributed Systems, facilitating efficient and reliable data transfer during live lectures.

The recognition pipeline follows modular design principles, allowing each stage (preprocessing, OCR, fusion, and storage) to operate independently while sharing defined data formats. This reflects software architecture and object-oriented design methods.

For the web interface, human-computer interaction (HCI) and UI design principles are applied to create a clear and accessible layout. Real-time synchronization uses event-driven programming techniques taught in Real-Time Systems.

Testing and validation follow structured methods from Software Validation, including unit, integration, and performance testing. Version control and reviews help maintain quality throughout development.

Overall, the project combines technical knowledge, teamwork, and engineering discipline to produce a complete and reliable software system that demonstrates the practical application of the knowledge we've gained through our program.

7. A proposed timetable for completion of the project including major intermediate milestones.

Component / Milestone	**Prerequisite**	**Assigned Members**	**Estimated Duration**	**Earliest Start**	**Latest Start**	**Earliest Finish**	**Latest Finish**
Frame Preprocessing	N/A	Eric C Sandy A	2 week	October 17, 2025	October 31, 2025	October 31, 2025	November 14, 2025
Content Segmentation	Frame Processing	Cameron BD Sandy A	2 week	October 31, 2025	November 14, 2025	November 14, 2025	November 28, 2025
Milestone 1 Capture board content with calibrated preprocessing for isolating and rectifying board images. Accurately segment the written areas of the processed board images as text, equations, and diagrams (drawings).							
Handwriting OCR	N/A	Sandy A Cameron BD	4 week	November 14, 2025	November 28, 2025	December 12, 2025	January 23, 2026
Milestone 2 Train and integrate the handwriting recognition module using fine-tuned TrOCR models based on LectureMath [3] and LectureVideoDB [4] datasets. Integrate with content segmentation to identify text with position data and confidence scores. Validate text extraction accuracy under classroom lighting and handwriting conditions.							

Math OCR	N/A	Cameron BD Eric C	3 week	November 14, 2025	December 5, 2025	December 5, 2025	January 23, 2026
Diagram Vectorization	N/A	Eric C Sandy A	3 week	November 14, 2025	December 5, 2025	December 5, 2025	January 23, 2026
Milestone 3 Finalize the Math OCR and Diagram Vectorization components. The Math OCR model will be fine-tuned using CROHME [6] and LectureMath [3] datasets to accurately convert handwritten equations into structured LaTeX. In parallel, the diagram module will transform drawn figures into clean vector representations using OpenCV and Potrace. Both outputs will include coordinate mapping for alignment with board regions, completing the visual recognition stage of the system.							
Speech-to-Text (ASR)	N/A	Sandy A Eric C	1 week	November 14, 2025	December 5, 2025	December 5, 2025	January 23, 2026
Multimodal Fusion and Structuring	OCR, Vectorization, ASR	Cameron BD Eric C Sandy A	3 week	December 12, 2025	January 23, 2026	January 30, 2026	February 13, 2026
Milestone 4 Complete the Multimodal Fusion and Structuring stage by aligning all recognized board content with transcribed speech from the ASR system. Develop the structured lecture data format used for both real-time streaming and post-lecture review. This milestone marks the first fully synchronized end-to-end lecture sequence, where text, equations, diagrams, and audio are time-aligned and searchable.							
AI Services (Summarization,Translation, Q&A)	Multimodal Fusion and Structuring	Sandy A Cameron BD	1 week	January 30, 2026	February 27, 2026	February 6, 2026	March 6, 2026
Website and Storage	Multimodal Fusion and Structuring	Eric C Cameron BD	3 week	January 30, 2026	February 13, 2026	February 20, 2026	March 6, 2026
Project Finalization and testing	All of the above	Eric C Cameron BD Sandy A	2 week	February 20, 2026	March 6, 2026	March 6, 2026	March 20, 2026
Milestone 5 Integrate all major system components, including AI services for summaries and translations, the website interface, and storage modules. Perform system wide testing, optimization, and final validation of performance and usability. Deliver a complete demonstration of ChalkStream's capabilities, showing real-time capture, recognition, synchronization, and playback.							

7.1 Frame Preprocessing:

- Prerequisite: None
- Number of people assigned: 3
- Estimated duration: 2 weeks
- Earliest Start: October 17, 2025
- Latest Start: October 31, 2025
- Earliest Finish: October 31, 2025
- Latest Finish: November 14, 2025

7.2 Content Segmentation
- Prerequisite: Frame Processing
- Number of people assigned: 3
- Estimated duration: 2 week
- Earliest Start: October 31, 2025
- Latest Start: November 14, 2025
- Earliest Finish: November 14, 2025
- Latest Finish: November 28, 2025

7.3 Handwriting OCR
- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 4 weeks
- Earliest Start: November 14, 2025
- Latest Start: November 28, 2025
- Earliest Finish: December 12, 2025
- Latest Finish: January 23, 2026

7.4 Math OCR
- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 3 week
- Earliest Start: November 14, 2025
- Latest Start: December 5, 2025
- Earliest Finish: December 5, 2025
- Latest Finish: January 23, 2026

7.5 Diagram Vectorization
- Prerequisite: Content Segmentation
- Number of people assigned: 1
- Estimated duration: 3 week
- Earliest Start: November 14, 2025
- Latest Start: December 5, 2025
- Earliest Finish: December 5, 2025
- Latest Finish: January 23, 2026

7.6 Speech-to-Text (ASR)
- Prerequisite: None
- Number of people assigned: 1
- Estimated duration: 1 week
- Earliest Start: November 14, 2025 (all members are busy on other tasks before then)
- Latest Start: December 5, 2025
- Earliest Finish: December 5, 2025
- Latest Finish: January 23, 2026

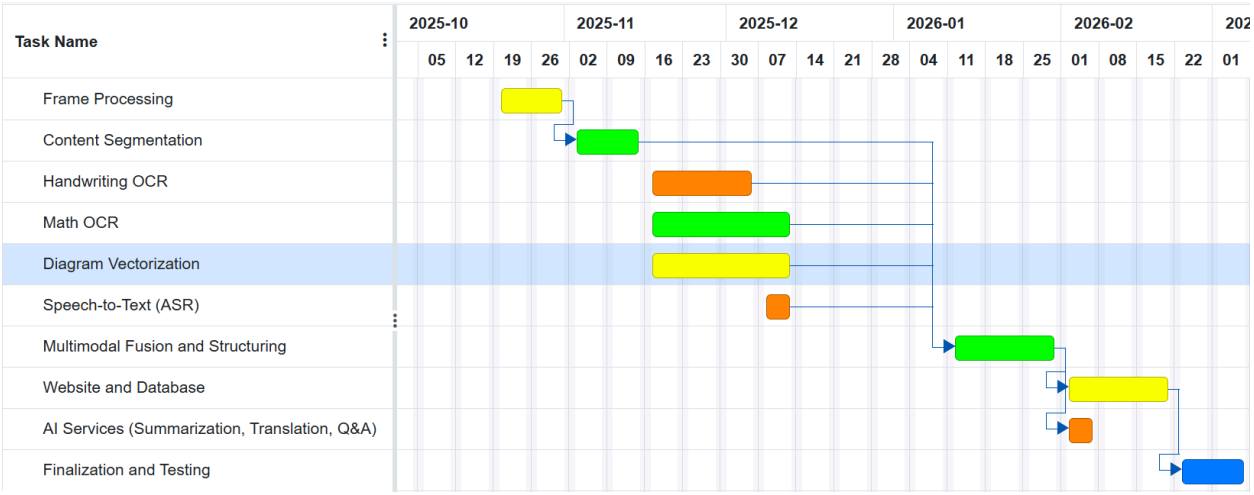
7.7 Multimodal Fusion and Structuring
- Prerequisite: OCR, Diagram Vectorization, Speech-to-Text
- Number of people assigned: 3

- Estimated duration: 3 week
- Earliest Start: December 12, 2025
- Latest Start: January 23, 2026
- Earliest Finish: January 30, 2026
- Latest Finish: February 13, 2026

7.8 AI Services (Summarization, Translation, Q&A)
- Prerequisite: Multimodal Fusion and Structuring
- Number of people assigned: 1
- Estimated duration: 1 week
- Earliest Start: January 30, 2026
- Latest Start: February 27, 2026
- Earliest Finish: February 6, 2026
- Latest Finish: March 6, 2026

7.9 Website and Storage
- Prerequisite: Multimodal Fusion and Structuring
- Number of people assigned: 3
- Estimated duration: 3 week
- Earliest Start: January 30, 2026
- Latest Start: February 13, 2026
- Earliest Finish: February 20, 2026
- Latest Finish: March 6, 2026

7.10 Project Finalization and testing
- Prerequisite: All of the above
- Number of people assigned: 3
- Estimated duration: 2 week
- Earliest Start: February 20, 2026
- Latest Start: March 6, 2026
- Earliest Finish: March 6, 2026
- Latest Finish: March 20, 2026



Yellow: Eric
Green: Cameron
Orange: Sandy
Blue: All

8. Project risks and mitigation strategies.

8.1. Error in pre-processing

Description: There can be errors in the pre-processing process which result in poorly processed images. (Ex. board cropped unevenly, script fails to capture all written portion) This cause the final output to be unstable
Risk likelyhood: Very High
Risk impact: Very low
Proposed Solutions:

- Test run requirement:
  - Professors are required to conduct a test run and adjust the sensitivity of the image crop, ensuring the board is correctly cropped.
- Frame Comparison:
  - Each processed frame will be compared with adjacent frames for crop alignment and writing alignment. Areas in frames that have a lot of variation over adjacent frames will be ignored

8.2. Error in AI process

Description: AI errors can occur which result in wrongly received drawing/writings.
Risk Likelihood: medium
Risk Impact: low
Proposed solution:

- Manual Selection
  - Students may manually select what type the output is (Diagrams/words/math equation) and the correct AI will transcribe the output
- Original option
  - Students may choose to use the pre-processed image as an option.

8.3. External API down

Description: External API may not be always active. The project may not work properly if the relied APIs are down.
Risk Likelihood: low
Risk Impact: Medium
Proposed Solution:

- Feature adaption
    - All procedures of the running project should be prepared to handle the absence of any API features. Quality is expected to be reduced, but all procedures should remain functional.
- API check
    - The system check the availability of all API, allowing the program to adapt beforehand
    - Features built on user UX will be greyed out if the API are not available

8.4. Server shutdown due to fatal error
Description: Servers shutdown unexpectedly due to fatal errors, rendering the project unavailable for the class and losing progress.
Risk Likelihood: very-low
Risk Impact: High
Proposed Solution:

- Emergency Save
    - The ongoing lecture will save the notes before the server fully shuts down. The program will have the ability to resume the lecture from saved files, allowing the lecture to still be saved in one file.

8.5. Project Progress falling behind
Description: Project may fall behind due to work efficiency or unexpected challenges.
Risk Likelihood: Medium
Risk Impact: Medium
Proposed Solution:

- Time table revise
    - Revise the timeline to account for the extra challenges of the project but ensure enough time was given for the upcoming projects. Group members will try their best to catch up on the schedule.
- Features cut
    - If catchup was not feasible, reduction in features would be necessary to ensure the project is delivered on time.

8.6. Members become unavailable during progress due to unforeseen circumstances
Description: Members of the project team may become unavailable temporarily or permanently for unforeseen circumstances, requiring the rest of the members to shoulder the task.
Risk Likelihood: Very low
Risk Impact: High to Critical
Proposed Solution:

- Have an overseer for all task with only 1 members working on
    - This allows the overseer to pick up the work and take control of the task if the responsible member suddenly becomes unavailable.
- Project scope rework
    - Rework the scope of the project by cutting features and rework on the timeline to ensure the delivery of the project could be on time even in the absence of one member.

8.7 Performance
Description: High latency in image processing due to camera feed lag or/and slow OCR/ML inference. Frame drops or lag if CPU can't keep up with continuous background updates. False positives when lighting changes or the instructor moves frequently.
Risk Likelihood: High
Risk Impact: High to Critical
Proposed Solutions:

- Asynchronous processing
    - Process model updates in a separate thread/process from stroke detection.
    - Offloads heavier work (background model updates, ML inference) so they don't block the real-time processes (preprocessing, differencing, stroke detection)
- Selective updates
    - Update the background model only in stable regions (areas without active writing or motion). Reduces unnecessary computation and prevents the model from misclassifying active regions (chalk strokes, instructor movement) as background.
- Hardware Acceleration
    - Use OpenCV's CUDA backend (GPU acceleration) for operations like filtering, thresholding, and background subtraction.
    - Offloads compute-heavy tasks to GPU, significantly improving frame rate and reducing latency.

8.8 Scalability
Description: Backend unable to handle a large number of concurrent student sessions. Server overloads when multiple users access synchronized lecture streams.
Risk Likelihood: Medium
Risk Impact: High
Proposed Solutions:

- WebSocket connections
    - Using WebSocket connections instead of traditional HTTP requests to send and receive data continuously between the backend and frontend

- Load balancing and microservice architecture (separate capture, recognition, and streaming modules)
    - Dividing your system into smaller, independent services, each responsible for a specific part of the workflow, and then distributing workload among multiple instances to handle more users or heavy processing smoothly

8.9 Reliability
Description: System components (e.g., capture module, recognition engine, or backend service) may fail mid-session, causing interruptions or lost data during live lectures.
Risk Likelihood: Medium
Risk Impact: High
Proposed Solutions:

- Health Checks and Auto-Restart
    - Implement watchdog threads or health-check APIs that periodically verify system component responsiveness.

9. Required special components and facilities that you require.

ChalkStream requires standard software tools and hardware that can be set up and managed by the project team. All components are open source or freely available online, and no specialized university facilities are needed at the time of writing the proposal.

The system will be developed using Python, with libraries such as OpenCV, NumPy, and PyTorch for image processing and recognition. The backend will use FastAPI with a PostgreSQL database, while the frontend will be built with React. Model testing and fine-tuning can be done locally on personal machines with GPU support or through cloud platforms like Google Colab when required.

All storage, deployment, and testing environments will be managed by the team using self-hosted or cloud-based services, ensuring full independence for development and demonstration.

10. References

[1] J. Howse and J. Minichino, Learning OpenCV 4 Computer Vision with Python 3, 3rd ed. Packt Publishing, 2020. (Available through Carleton Omni)

[2] Baek, Y., Lee, B., Han, D., Yun, S., & Lee, H. (2019). Character Region Awareness for Text Detection (CRAFT). In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2019), 9365-9374.(Available online)

[3] Dávila, K. LectureMath: Methods and Datasets for Automated Lecture Video Analysis — code + data for FCN-LectureNet (IEEE Access 2021). GitHub repository, 2021. GitHub

[4] Dutta, K., Mathew, M., Krishnan, P., & Jawahar, C. V. Localizing and Recognizing Text in Lecture Videos (LectureVideoDB Dataset). ICFHR 2018. (Available) (Hugging face dataset)

[5] D. W. Lee, C. Ahuja, P. P. Liang, S. Natu, & L.-P. Morency, Lecture Presentations Multimodal Dataset: Towards Understanding Multimodality in Educational Videos, 2023. (Available)

[6] Xie, Y., Mouchère, H., Liwicki, F. S., Rakesh, S., Saini, R., Nakagawa, M., Nguyen, C. T., & Truong, T.-N. (2023). ICDAR 2023 CROHME: Competition on Recognition of Handwritten Mathematical Expressions (Dataset). (Available)

[7] Radford, A., Kim, J.W., Xu, T., Brockman, G., Mcleavey, C. & Sutskever, I.. (2023). Robust Speech Recognition via Large-Scale Weak Supervision. Proceedings of the 40th International Conference on Machine Learning, in Proceedings of Machine Learning Research 202:28492-28518 Available from https://proceedings.mlr.press/v202/radford23a.html.